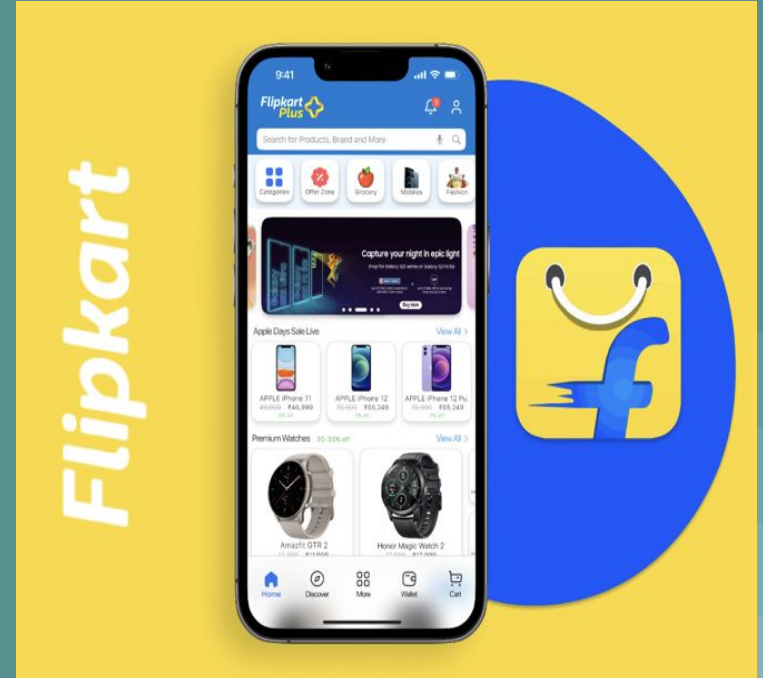


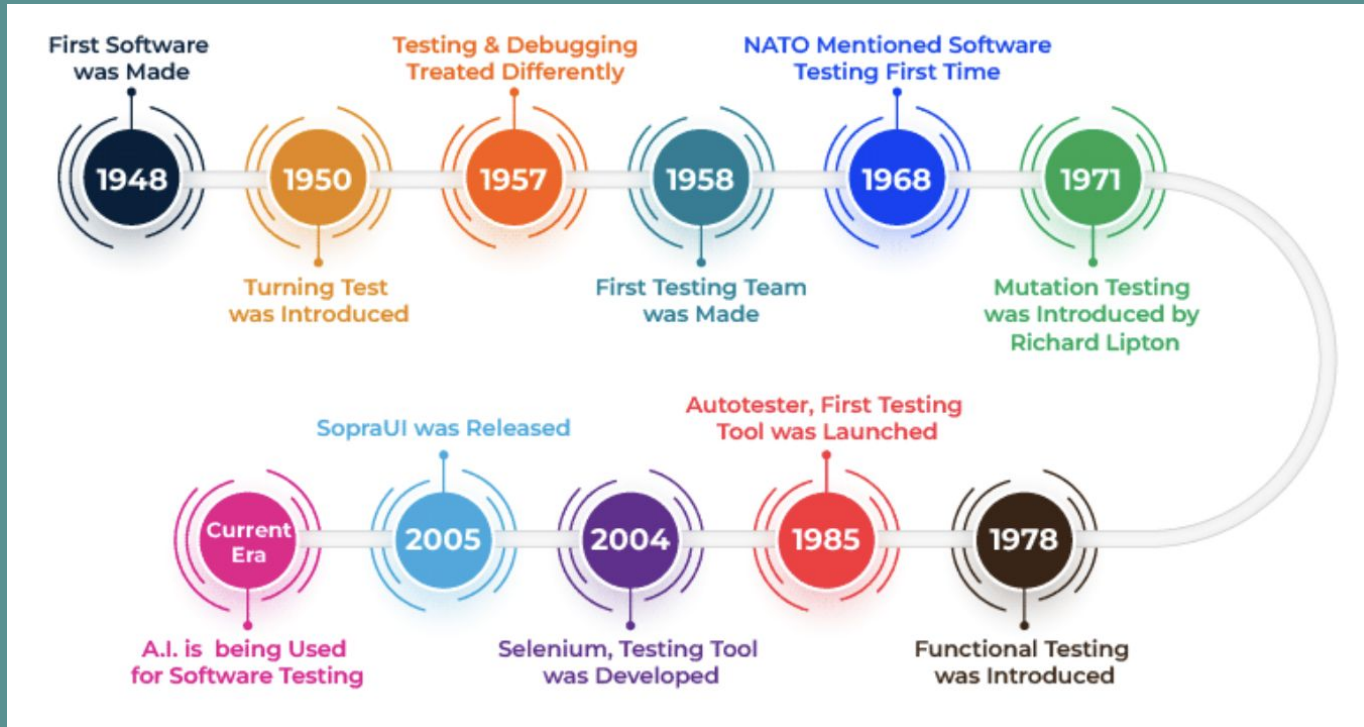
QA & Automation

Keshav Potdar

Why we need Testing?



Software Testing History



Key Aspects

- Accuracy
 - Consistency
 - Security
 - Performance
 - ROI (Maintainability, Optimization, Configurability, Recommendation)
 - UX
-
- Scalability
 - Compatibility
 - Configurability
 - Optimization (of Cost)

QA Methodologies

- Manual
 - Functional Testing
 - Unit Testing
 - Integration
 - System Testing
 - Internationalisation & Globalisation
 - Regression & Retesting
 - Acceptance
 - Alpha & Beta
 - Usability
 - Exploratory Testing
 - Monkey Testing
 - Fuzzy Testing
- Database Testing
- Security Testing
- API Testing
- Compatibility Testing
- Performance
 - Load Testing
 - Endurance Testing
 - Volume Testing
 - Spike Testing
- Automation
 - API
 - UI (Portal, Cross-Browser)
 - Mobile Apps

QA Methodologies - Practical

- Manual

- Functional Testing
- Unit Testing
- Integration
- System Testing
- Internationalisation & Globalisation
- Regression & Retesting
- Acceptance
- Alpha & Beta
- Usability
- Exploratory Testing
- Monkey Testing
- Fuzzy Testing

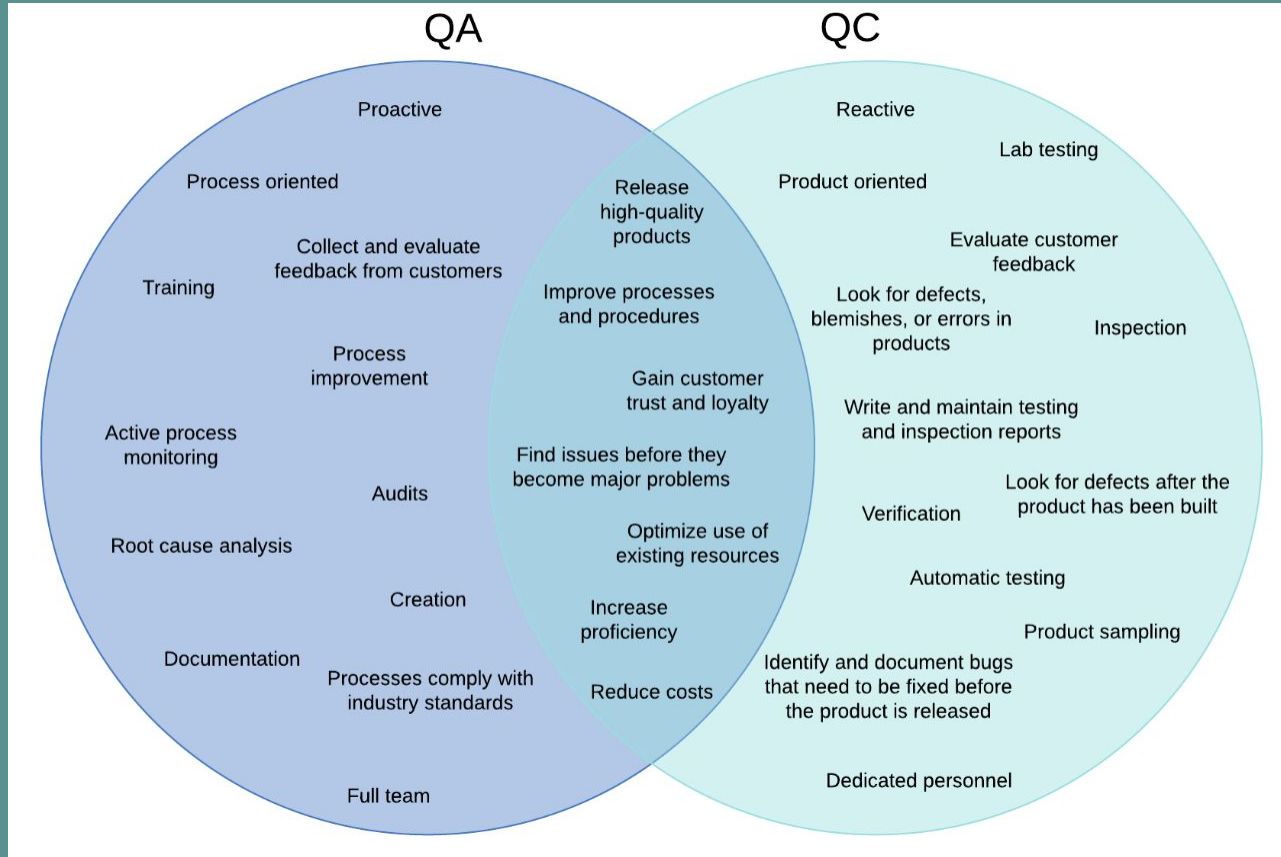


- Database Testing
- Security Testing
- API Testing
- Compatibility Testing
- Performance
 - Load Testing
 - Endurance Testing
 - Volume Testing
 - Spike Testing
- Automation
 - API
 - UI (Portal, Cross-Browser)
 - Mobile Apps

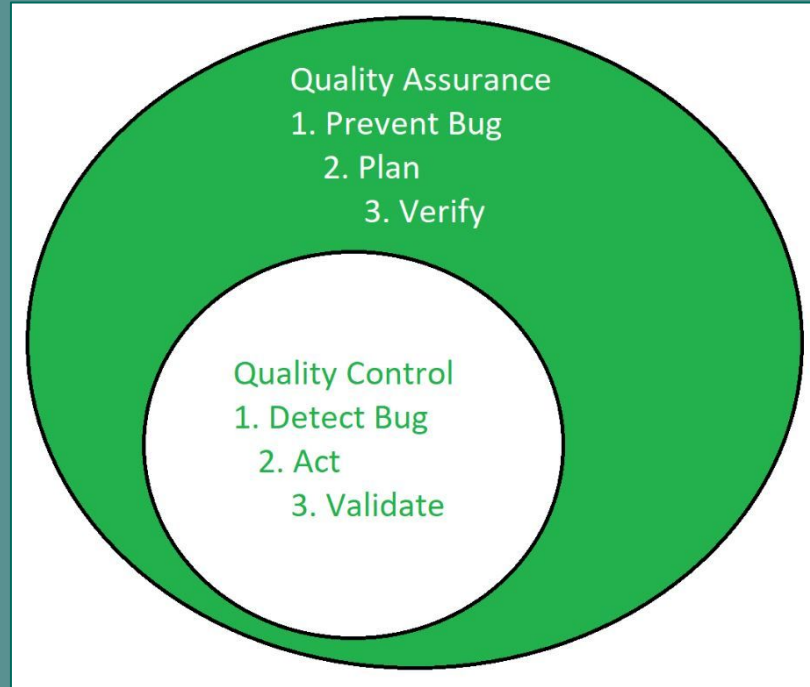
Verification & Validation

Verification	Validation
Are we building software product RIGHT?	Are we building RIGHT software product?
Usually No need to execute code	Usually need to execute code
Static Approach, Eg. Review, Walkthrough, etc	Dynamic Approach, Eg. Functional Testing, Regression, etc

QA vs QC



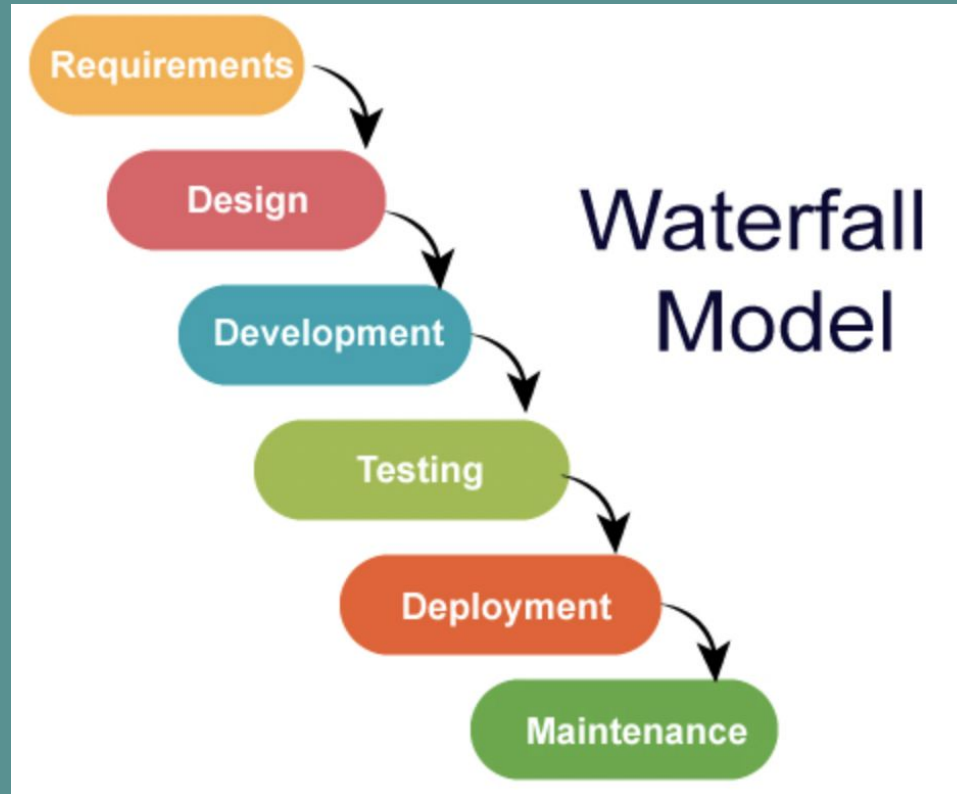
QA vs QC



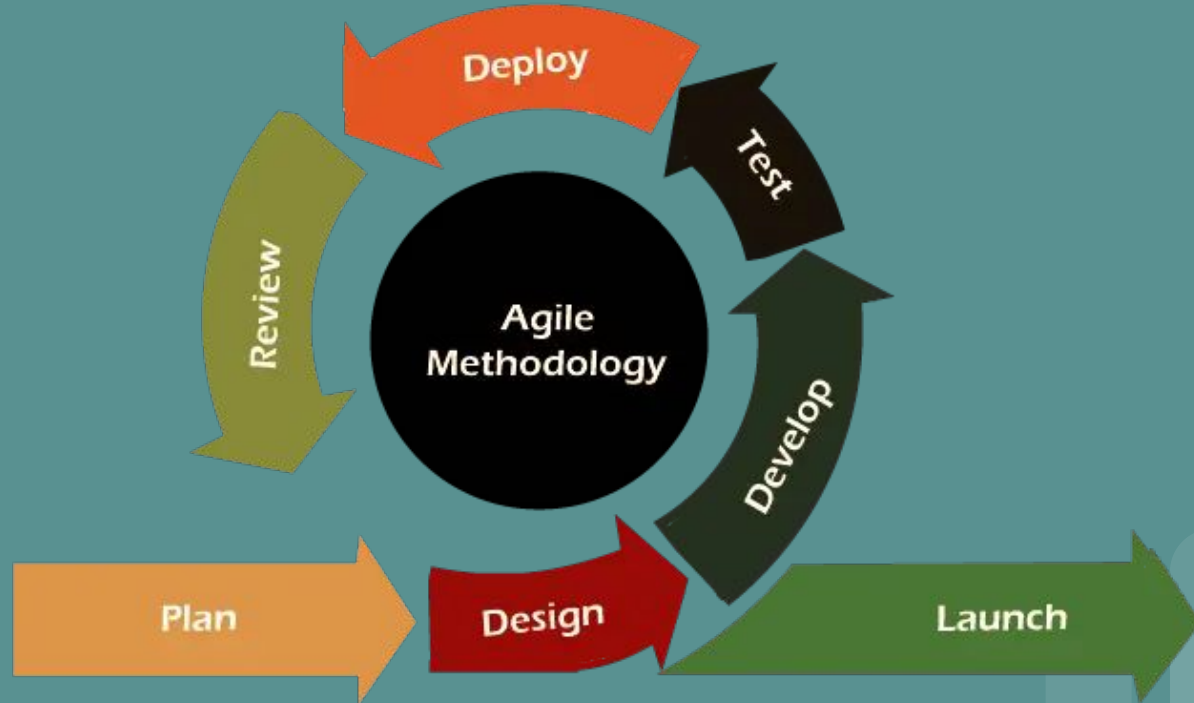
Testing Concepts

- Use Case
- Test Case
- Modules
- Bug
- Defect
- Fault
- Test Plan
- Testing Strategy
- Traceability Matrix
- Entry Criteria
- Exit Criteria
- AUT
- RoI
- LoE
- SDLC
- STLC
- Defect Life Cycle
- Bug Triaging
- Failover Mechanism
- BlackBox Testing
- WhiteBox Testing
- GreyBox Testing

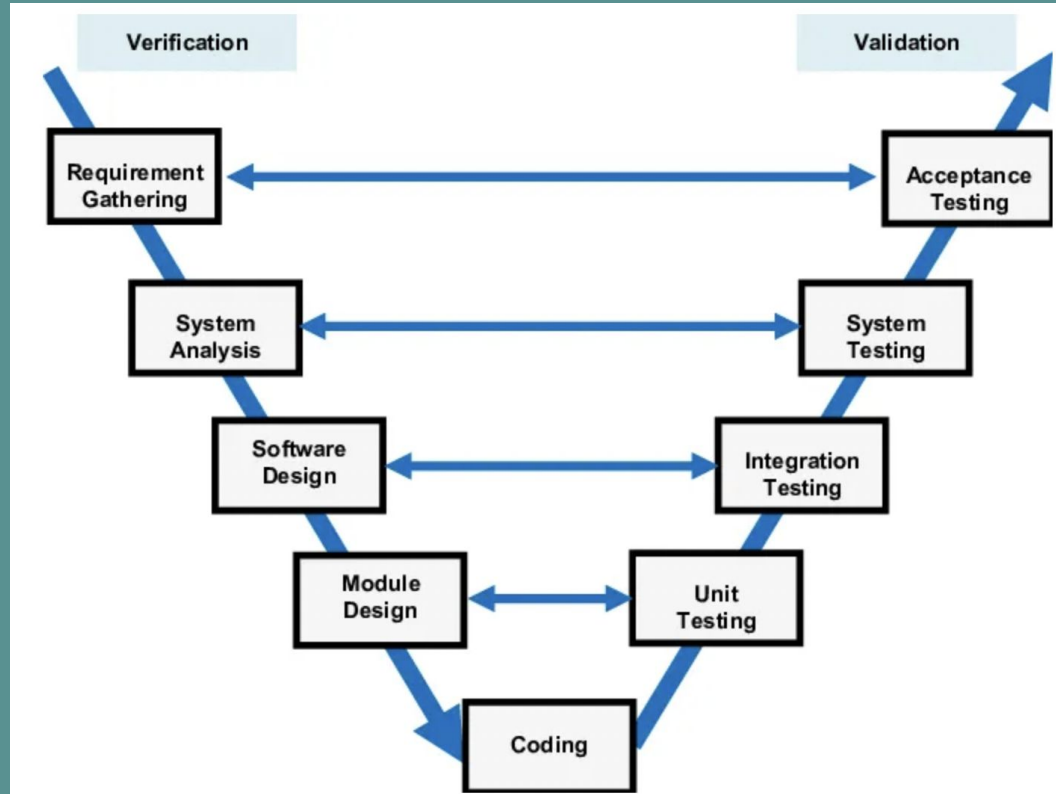
Waterfall Model



Agile Model



Verification & Validation Model



Case Study



Let's Do It Together...

Test Cases - Reynold Ball Pen - Pointers-1

Functional Test Cases :

- Verify that the pen writes smoothly on normal paper.
- Verify ink flows immediately when writing starts.
- Verify the ball rotates freely and dispenses ink evenly.
- Verify the pen can write continuously without breaks in ink.
- Verify pen writes in different directions (upwards, sideways, upside down).
- Verify cap (if present) fits securely and prevents ink drying.
- Verify retractable mechanism (if applicable) works properly.
- Verify refill is replaceable (if design allows)....

Test Cases - Reynold Ball Pen - Pointers-2

Usability Test Cases :

- Verify grip is comfortable for long writing sessions.
- Verify pen weight is manageable (not too heavy/light).
- Verify balance of the pen allows easy writing.
- Verify the pen does not leak ink when carried in a pocket.
- Verify pen body material is smooth and does not cause discomfort.
- Verify clip (if present) holds securely on a pocket or notebook.
- Verify writing readability on different paper qualities (rough, glossy, recycled)...

Test Cases - Reynold Ball Pen - Pointers-3

Performance Test Cases :

- Verify how many pages the pen can write before running out of ink (ink life)
- Verify writing consistency for long durations (no skipping)
- Verify drying speed of ink
- Verify line thickness remains consistent throughout usage
- Verify pen writes well under varying pressures (light, medium, heavy)
- Verify pen works in different temperatures (hot, cold environments)
- Verify continuous writing test (e.g., 10 pages nonstop) does not damage the tip....

Test Cases - Reynold Ball Pen - Pointers-4

Reliability Test Cases :

- Verify pen does not leak ink when shaken.
- Verify pen does not break when dropped from desk height.
- Verify pen cap clip does not snap under normal use.
- Verify pen writes even after being unused for several days.
- Verify pen ink does not dry quickly if left uncapped for a short period (1-2 mins).
- Verify pen ink color remains consistent until exhaustion.....

Test Cases - Reynold Ball Pen - Pointers-5

Branding (UI) Test Cases :

- Verify branding position on pen is correct
- Verify branding color & size is correct and as per PRD design
- Verify pen branding/logo is visible and not faded.
- Verify no other parts of pen shadowing branding logo or name
- Verify logo, branding name is easy to read and visible under daylight and low light conditions
- Verify endurance of logo branding name, how long it stays and not disappeared from pen
- Verify there are no spelling errors in the branding text or labeling

Test Cases - Reynold Ball Pen - Pointers-6

Negative Test Cases :

- Verify pen does not leak when stored horizontally/vertically.
- Verify writing is not possible when refill is removed.
- Verify ink does not spread or blot when pressing hard.
- Verify pen does not crack when moderate force is applied while writing.
- Verify pen does not cause paper tearing under normal pressure.
- Verify cap does not loosen after repeated usage.

Test Cases - Reynold Ball Pen - Pointers-7

Stress & Edge Test Cases :

- Verify if the pen writes on non-standard surfaces (cardboard, tissue, plastic).
- Verify pen behavior when left in direct sunlight.
- Verify writing after dropping pen in water (if possible).
- Verify if pen works after being dropped multiple times.
- Verify ink does not dry out in extremely cold conditions.
- Verify if no sharp edge/s or pointer to pen body or cap
- Verify if pen writes when user writes it with volume pressure on it

Test Cases - Reynold Ball Pen - Pointers-8

Alpha & Beta Test Cases :

- Verify if the pen writing functionality at sand box simulating client environment
- Verify if effect on pen ink at client location
- Verify if the pen writes at client location
- Verify impact on pen body at client location
- Verify threading at client location
- Verify durability performance of pen (how long it writes) at client location

Test Cases Format

TC ID	TC Title	Prerequisite	Test Data	Test Steps	Expected Result	Test Status	Req ID	Defect ID/s

Test Case & Defect Management Tools Eg

TEST CASE MANAGEMENT TOOLS EG :

- Zephyr (JIRA plugin)
- Test Rail
- Testlink
- QAtouch
- Xray
- QC

DEFECT MANAGEMENT TOOLS EG :

- JIRA
- BugZilla
- Mantis BT
- Zoho Bug Tracker

Bug Example - JIRA

<https://www.youtube.com/watch?v=X2yYnADRYTw>

Sample bug

 Attach  Add a child issue  Link issue  

Description

Preconditions:

Mention here any steps that need to be performed to get the environment ready.

Steps to reproduce:

1. The first step
2. The second step

Expected result:

What should happen

Actual result:

What really happens

Environment

Keshav Potdar
PRODUCTION

To Do ▾

Details

Severity	High
Priority	 High
Assignee	 Unassigned
Labels	 API
Due date	Feb 28, 2022
Reporter	 Andreea Draniceanu
Automation	 Rule executions

Created 2 minutes ago

Updated 2 minutes ago

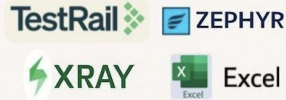


Configu

TESTING TYPES AND TOOLS

MANUAL TESTING

Testing without automation tools



AUTOMATION TESTING

Using scripts/tools to automate tests



PERFORMANCE TESTING

Checks system behavior under load



SECURITY TESTING

Identifies vulnerabilities in an application



USABILITY TESTING

Evaluates user experience and ease of use



COMPATIBILITY TESTING

Ensures app works across browsers device, OS



API TESTING



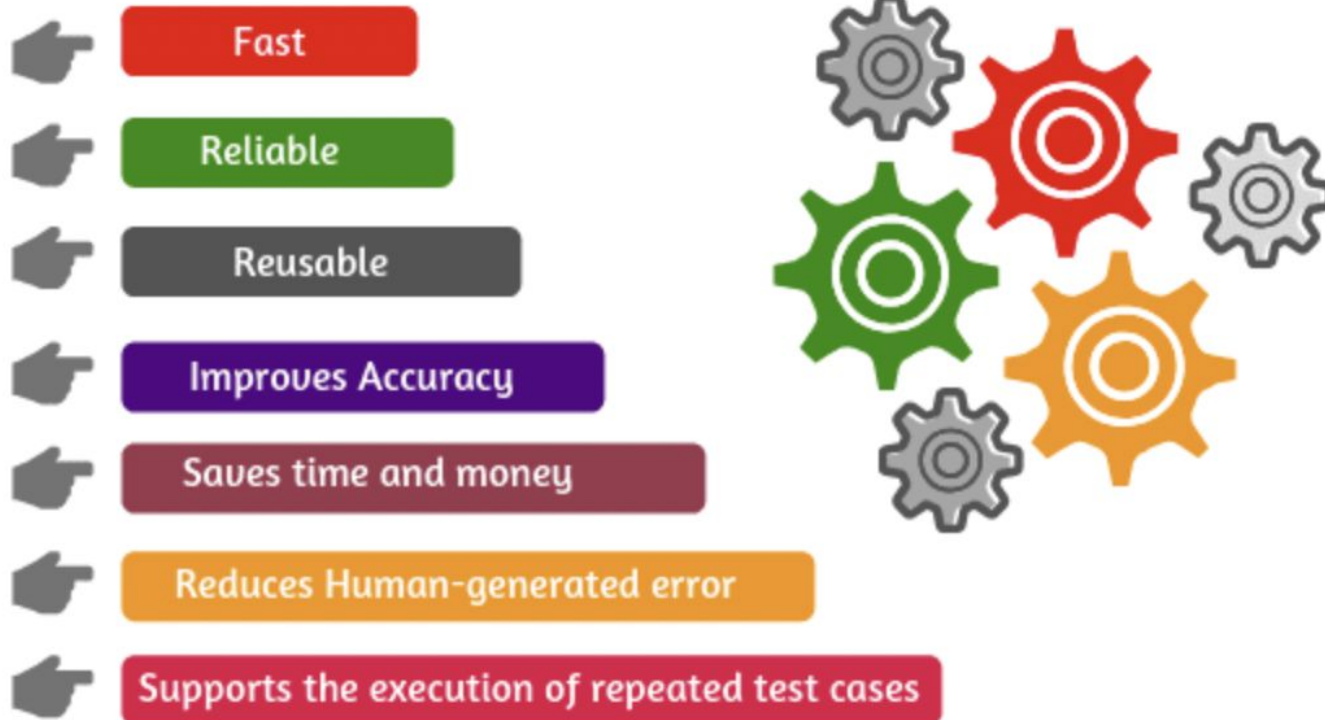
CI/CD TOOLS



What is Automation ?



Why Automation ?



Automation Framework Types

Page Object Model (POM) Framework

- Creates Object Repository for UI Elements
- Populer for UI Automation

Data Driven Framework

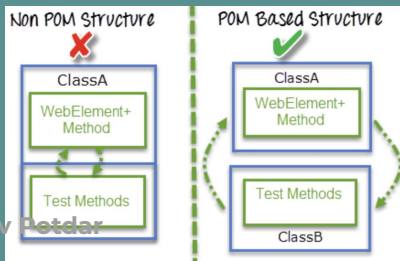
- Input will be fetched from external sources like DB or separate file like Excel, etc
- Output will be stored separately

Keyword Driven Framework

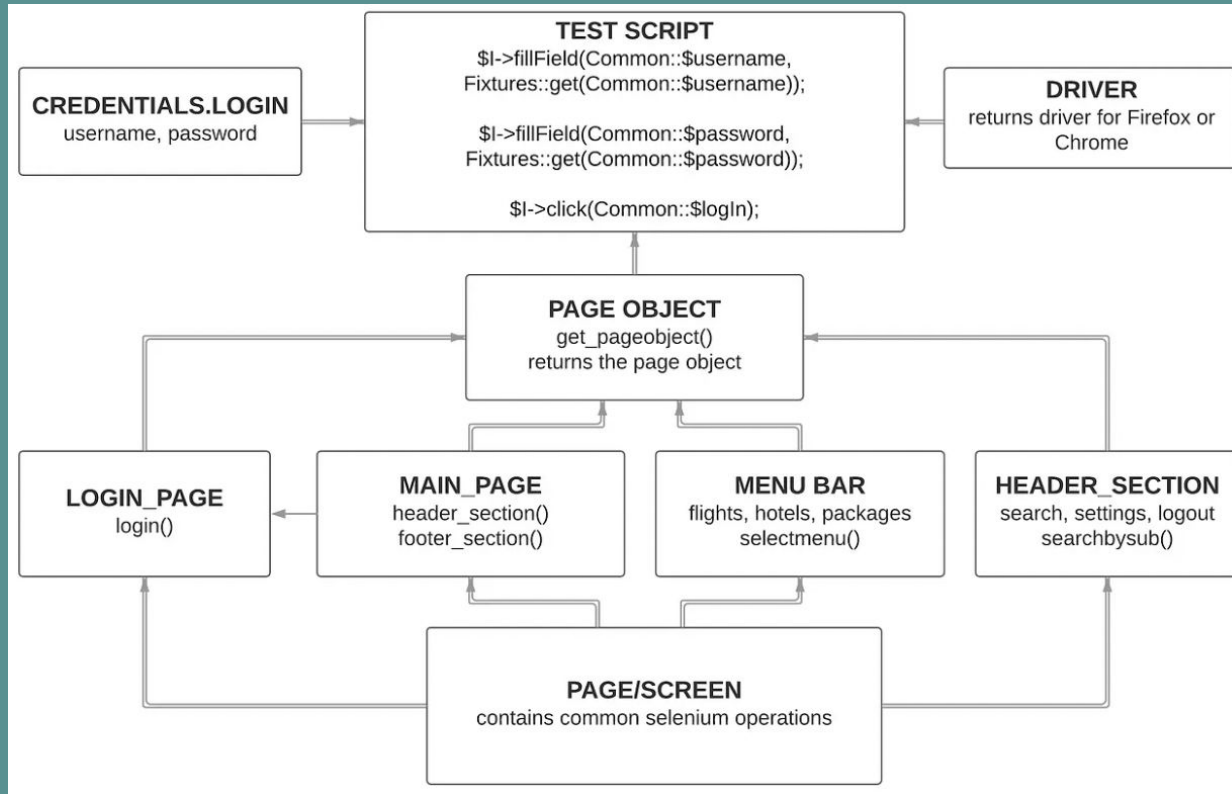
- Application independent framework
- Uses keywords to explain actions to be performed on AUT

Hybrid Framework

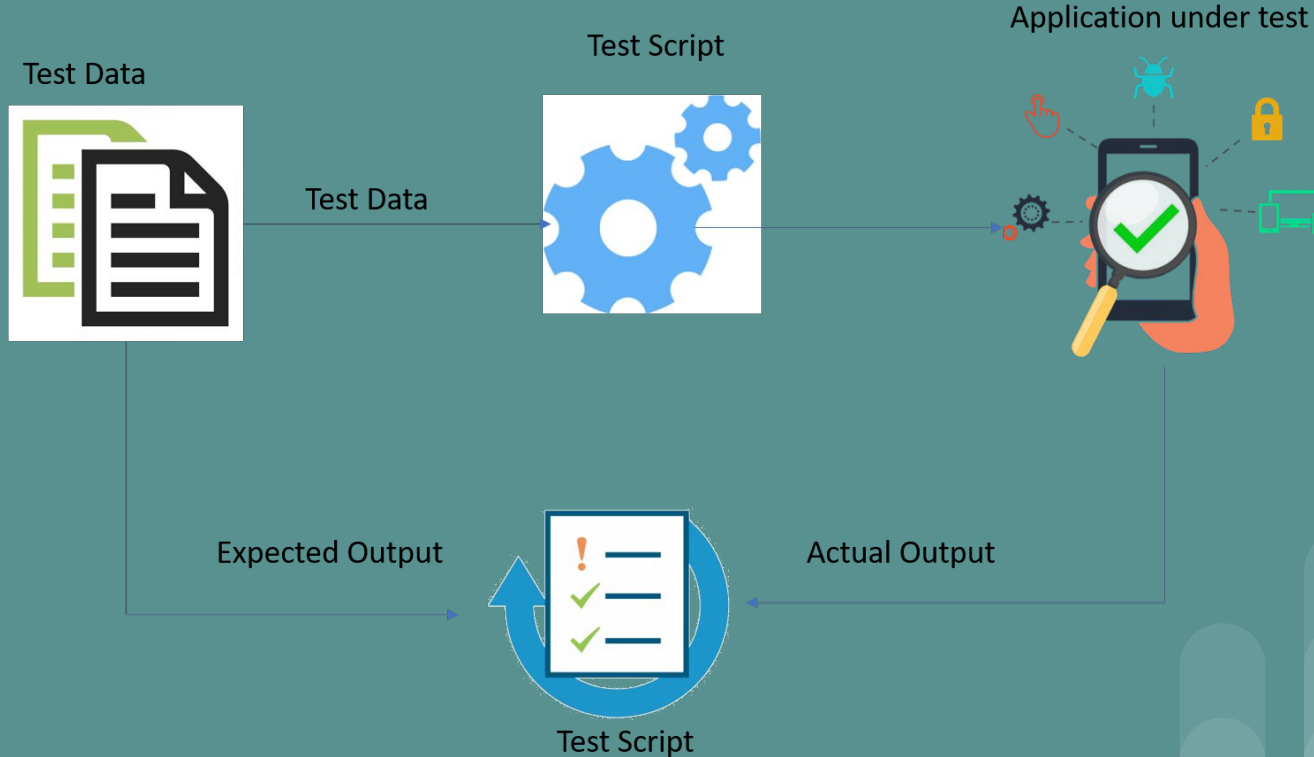
Moduler Framework



POM Framework



Data Driven Framework



Automation Tool Examples





Automation using Selenium

Keshav Potdar

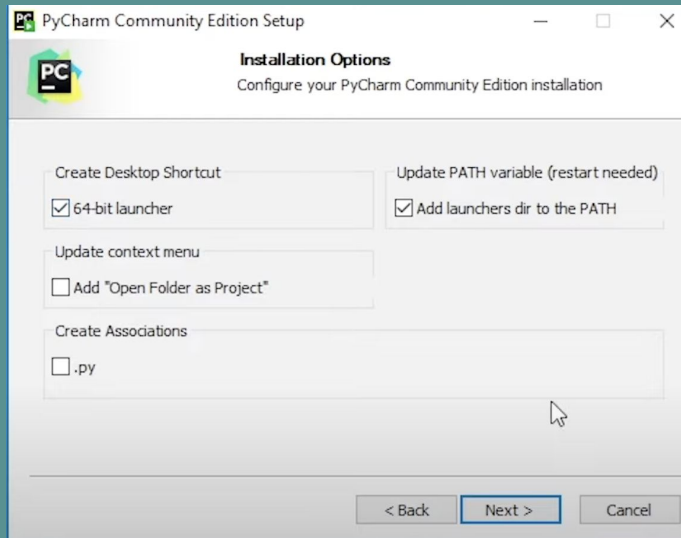


Agenda

- What is Selenium ?
- Selenium Components
 - Selenium IDE
 - Selenium RC
 - Selenium Grid
 - Selenium webdriver
- Selenese
- How Selenium works?
- Pros & Cons of Selenium
- Automation using Selenium
- POM Design & Selenium

Prerequisite

- Install PyCharm Community Edition
 - Downlink Link
 - For Windows : <https://www.jetbrains.com/pycharm/download/?section=windows>
 - For Mac : <https://www.jetbrains.com/pycharm/download/?section=mac>
 - Download any latest or preferable version 2023.2.2
 - During installation, ensure to select mentioned checkbox->



Prerequisite - 2

- Create Python Virtual Environment
 - Goto PyCharm Terminal: `pip3 install virtualenv`
 - Create new folder and goto this new folder using PyCharm terminal using “cd” command
 - `virtualenv myvirtualenv` or `python -m venv <new_folder_path>`
 - `source <new_folder_path>/bin/activate`
 - `virtualenv --version`
- Install Python v3.12.0
 - Preferable way: Open PyCharm Terminal -> Ensure Virtual Env is activated as above → `pip3 install python`
 - Manual Downloader For Windows: <https://www.python.org/downloads/release/python-3120/>
 - Manual Downloader For Mac:
 - Download Homebrew first: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
 - In Terminal run command: `brew install python`
 - Confirm installation using running command “python3 --version” in Terminal or CMD

Prerequisite - 3

- Install Java v11.0.24
 - Download URL : <https://www.oracle.com/java/technologies/downloads/>
 - Set JAVA_HOME path in Environment Variables→System Variables
 - Add Java to System Path
 - Ensure Java installed correctly using “java --version” command in terminal / cmd
- Install Chromium (Test Automation Browser)
 - Download URL: <https://storage.googleapis.com/chrome-for-testing-public/128.0.6613.119/mac-x64/chrome-mac-x64.zip>
- Add ‘SelectorsHub’ Chrome plugin :
 - <https://chrome.google.com/webstore/search/SelectorsHub> → Add to Chrome → Restart Chrome → On Developer’s Tool click right panel “>” arrows to locate installed SelectorsHub

Prerequisite - 4

- Install Mozilla Firefox Browser (if requires)
 - Download URL: <https://www.mozilla.org/en-US/firefox/browsers/windows-64-bit/>
- Install Selenium ChromeDrivers
 - Download URL: <https://chromedriver.storage.googleapis.com/index.html?path=114.0.5735.90/>
 - Save this ChromeDriver and note down file path of it
- Install GeckoDriver
 - Download URL: <https://github.com/mozilla/geckodriver/releases/tag/v0.35.0>

What is Selenium?

- Its an interface which interacts with browser to execute specified commands
- Contains open-source API collection.
- Supports multiple programming languages, including Python, Java, C#, Ruby, and JavaScript
- Created by Jason Huggins in 2004, an engineer in Thoughtworks

Selenium IDE

- IDE = Integrated Development Environment
- Record & Playback tool
- Available for Mozilla Firefox, Chrome browser, Microsoft Edge
- It runs recorded script using Selenium RC

Selenium RC

- RC = Remote Control
- OFFICIALLY DEPRECATED in v4 onwards
- As NO need for mediator between Browser Driver communicating to Real Browser, all its capabilities are merged into Selenium Webdriver
- RC was using javascript command for communication with real browsers

Selenium Grid

- Helps in achieving parallel execution
- Runs multiple test cases on multiple browser, on multiple machines (Node) by Hub on heterogeneous environment
- Sends webdriver commands to remote browser instance

Selenium Webdriver

- An **interface** which interacts with browser to execute specified commands
- **SELENIUM SCRIPT [Client]-----W3C Protocol---->BROWSER DRIVERS [Server]<----HTTP---->REAL BROWSERS**

Webdriver Language Bindings

Browser already follows W3C protocol

Python

Chrome Driver

Java

Gecko Driver

C#

Opera Driver

JavaScript

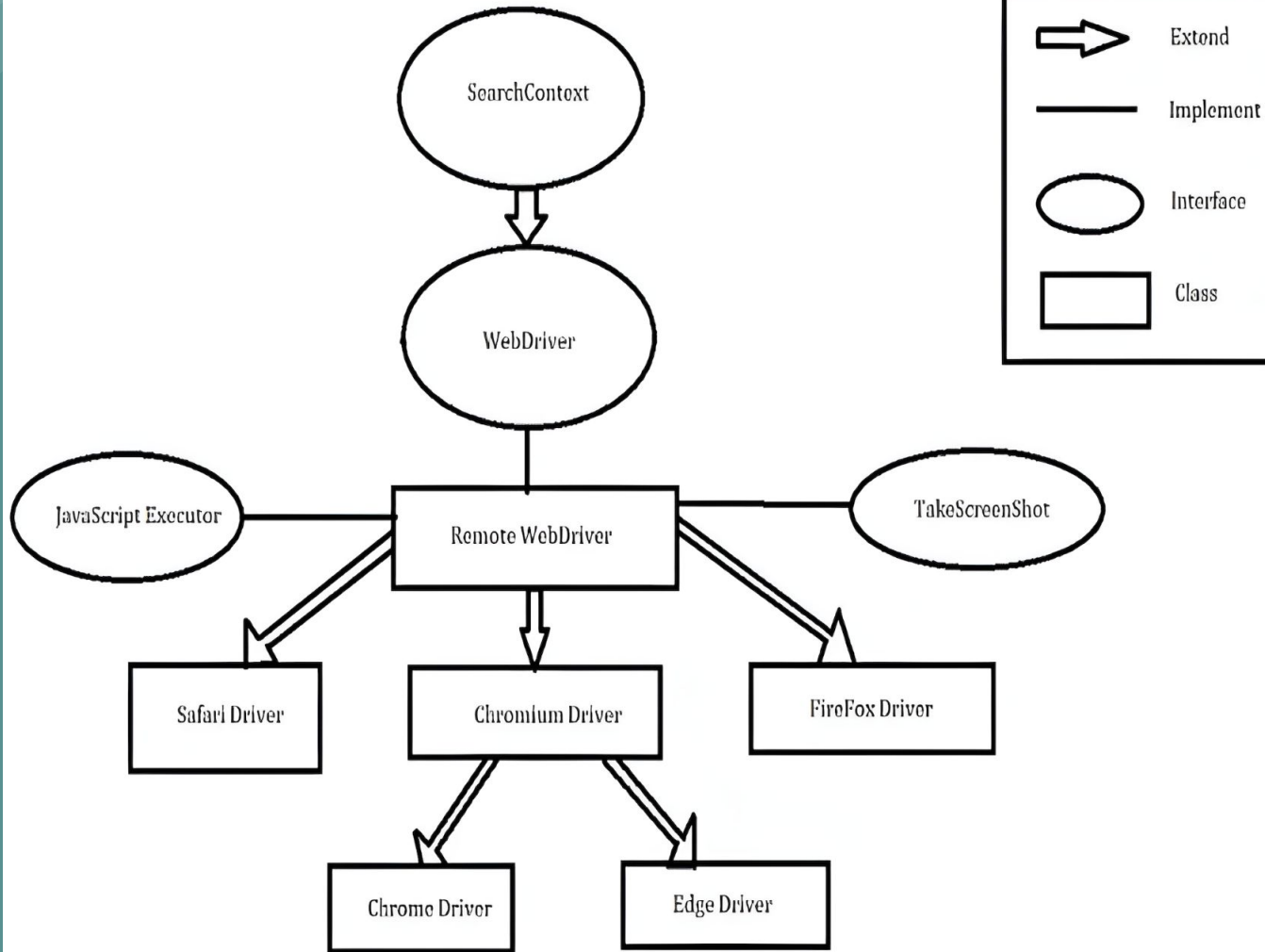
Edge Driver

Ruby

Safari Driver

- Due to using W3C protocol, communication with browsers are now more faster. Also, browser specific code not need which was majorly handled internally by selenium before communicating to browser.
- Selenium4.11 onwards, Selenium has stopped support for JSON wire protocol completely, which was their till v3.8. Also few major changes like removal of executable_path, version arguments from ChromeDriverManager
- Webdriver is an interface.
 - Search Context is topmost interface in Selenium API. Selenium Webdriver extends Search Context. Search Context has 2 main methods - findElement(), findElements()
 - Selenium v4 introduced Relative Locators like (above, below, toRightof,toLeftof, near) to identify surrounding element. It was not thr in v3

WEBDRIVER ARCHITECTURE



Selenium Advantage

- Saves testing time, specially regression cases, implies saving money and manual testing efforts
- Open source
- Supports multiple programming languages
- Supports multiple operating systems
- Perform Cross Browser Testing
- Allows simultaneous running of tests in various web browsers, environments and machines.
- Large helping community

Selenium Disadvantage

- Supports only webapp, does not support native apps like windows app, mobile apps (android, ios app) etc
- Does not have in-built report functionality
- Capta, Barcode reader are limitations
- Difficult for image testing
- User need to know programming language to use it

Selenium Drivers

ChromeDriver	Used to control Google Chrome Browser	<pre>from selenium import webdriver from selenium.webdriver.chrome.service import Service from selenium.webdriver.chrome.options import Options from webdriver_manager.chrome import ChromeDriverManager chrome_binary_path = "/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" options = Options() options.binary_location = chrome_binary_path service = Service(ChromeDriverManager(version="114.0.5735.90").install()) driver = webdriver.Chrome(service=service, options=options) driver.get("https://www.google.com")</pre>
GeckoDriver	Used to control Firefox Browser	<pre>from selenium import webdriver from webdriver_manager.firefox import GeckoDriverManager from selenium.webdriver.firefox.service import Service from selenium.webdriver.firefox.options import Options firefox_binary_path = "/Applications/Firefox.app/Contents/MacOS/firefox" options = Options() options.binary_location = firefox_binary_path service = Service(GeckoDriverManager().install()) driver = webdriver.Firefox(service=service, options=options) driver.get("https://www.google.com")</pre>

Selenium Drivers -2

OperaDriver	- Used to control the Opera Browser - OperaDriverManager removed from Selenium 4+ version as Opera is no more maintaining separate Opera driver. Achieve this using ChromeDriverManager	<pre>from selenium import webdriver from selenium.webdriver.chrome.service import Service from selenium.webdriver.chrome.options import Options from webdriver_manager.chrome import ChromeDriverManager options = Options() # Path to Opera browser binary options.binary_location = "/Applications/Opera.app/Contents/MacOS/Opera" driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options) driver.get("https://www.google.com")</pre>
EdgeDriver	Used to control the Microsoft Edge Browser	<pre>from selenium import webdriver from webdriver_manager.microsoft import EdgeChromiumDriverManager driver = webdriver.Edge(EdgeChromiumDriverManager().install())</pre>
SafariDriver	-Used to control Safari Browser -Safari limited to Apple OS only -SafariDriver is pre-installed with Safari on macOS, just enable	<pre>from selenium import webdriver # No need to explicitly import any driver as it comes pre-installed on macOS</pre>

Selenium Drivers -3

HTMLUnit

- Headless browser automation tool
- Without GUI
- Faster
- Mostly used in market for CI CD pipeline like Jenkins job suite execution

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager
```

```
chrome_options = Options()
chrome_options.add_argument("--headless") # Enable headless mode
chrome_options.add_argument("--start-maximized") # Start maximized
chrome_options.add_argument('--incognito')
chrome_options.add_argument('--window-size=1280,720')
chrome_options.add_argument('--incognito')
```

```
#Invoke existing chrome browser instance
driver=webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=chrome_options)
```

```
#Invoke specific chrome browser version instance
driver=webdriver.Chrome(service=Service(ChromeDriverManager(driver_version="127.0.6533.120").install()), options=chrome_options)
```

DOM

- Document Object Model
- Crucial in Web Development
- Represents document as a tree of objects
- Example:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is a paragraph.</p>
  <button id="myButton">Click Me!</button>
</body>
</html>
```

Document

```
└─ html
    ├── head
    │   └─ title
    │       └─ "My Page"
    └─ body
        ├── h1
        │   └─ "Hello, World!"
        ├── p
        │   └─ "This is a paragraph."
        └─ button
            └─ "Click Me!"
```

Selenium Locators

- Used to identify and locate web element on web page
- Need to perform specified action on those web element
- Various methods to locate web element
- Choosing the right locator type depends on the structure of the HTML and the attributes available for the elements you want to interact with

Locators Type

- **ID Locator**

- Description: Locates an element by its unique `id` attribute.
- Usage: Fastest and most reliable methods. Use it whenever available.
- Syntax: `element = driver.find_element(By.ID, "element_id")`
- Example: `element = driver.find_element(By.ID, "pass")`

- **Name Locator**

- Description: Locates an element by its `name` attribute.
- Usage: Useful for form elements like input fields.
- Syntax: `element = driver.find_element(By.NAME, "element_name")`
- Example: `element = driver.find_element(By.NAME, "pass")`

- **Class Name Locator**

- Description: Locates elements by their `class` attribute.
- Usage: Can be used to find elements with a specific class. Can select multiple elements with the same class.
- Syntax: `element = driver.find_element(By.CLASS_NAME, "class_name")`
- Example: `element = driver.find_element(By.CLASS_NAME, "inputtext")`

Locators Type - 2

- **Tag Name Locator**

- Description: Locates elements by their HTML tag name.
- Usage: Useful for finding elements like `<div>` , `<a>` , etc, ie element using their tag
- Syntax: `element = driver.find_element(By.TAG_NAME, "tag_name")`
- Example: `element = driver.find_element(By.TAG_NAME, "input")`

- **Link Text Locator**

- Description: Locates a link (anchor tag) by the exact text within the link.
- Usage: Useful for finding hyperlinks. Limited to anchor tag.
- Syntax: `element = driver.find_element(By.LINK_TEXT, "Exact Link Text")`
- Example: `element = driver.find_element(By.LINK_TEXT, "Forgotten password?")`

- **Partial Link Text Locator**

- Description: Locates a link by a portion of the text within the link.
- Usage: Useful when the full text is not known. Can match partial text from anchor tag.
- Syntax: `element = driver.find_element(By.PARTIAL_LINK_TEXT, "Partial Text")`
- Example: `element = driver.find_element(By.PARTIAL_LINK_TEXT, "Forgotten")`

Locators Type - 3

- **CSS Selector Locator**

- Description: Locates elements using a CSS selector.
- Usage: Similar to XPath but can be more concise for certain selections.
- Syntax: `element = driver.find_element(By.CSS_SELECTOR, "tag[attribute='value']")`
- Example: `element = driver.find_element(By.CSS_SELECTOR, "input[name='pass']")`

- **Xpath Locator**

- Description: Locates elements using an XPath expression.
- Usage: Very powerful and flexible; can navigate through the DOM structure.
- Syntax: `element = driver.find_element(By.XPATH, "//tag[@attribute='value']")`
- Example: `element = driver.find_element(By.XPATH, "//input[@name='pass']")` → Relative Xpath (using name attribute, starts with //, Slow)
- Example: `element = driver.find_element(By.XPATH, "/html/body/div[1]/div[2]/div/div[1]/div/form/div[2]/input")` → Absolute XPath (Full path from root, starts with /, Fast)

Xpath Sample -Table HTML

```
<tr class="row-21" xpath="1">  
  <td class="column-1">21.1</td>  
  <td class="column-2">21.2</td>  
  <td class="column-3">21.3</td>  
  ...  
  <td class="column-45">21.45</td>  
  <td class="column-46">21.46</td>  
  <td class="column-47">21.47</td>  
  <td class="column-48">21.48</td>  
  <td class="column-49">21.49</td>  
  <td class="column-50">21.50</td>  
</tr>
```

Question] Write XPATH to get row=21, column=46?

```
//tr[contains(@class,  
'row-21')]/td[normalize-space()='21.46']
```

Xpath Sample - Table with ~Text

Sample HTML:

```
<table>
```

```
<tr>
```

```
<td>21.46 </td>
```

```
<td>21.46</td>
```

```
<td>21.46</td>
```

```
<td>21.46</td>
```

```
</tr>
```

```
</table>
```

—> After applying `normalize-space()` , becomes "21.46" . matches the condition.

—> After applying `normalize-space()` , remains "21.46" . matches the condition.

—> After applying `normalize-space()` , becomes "21. 46" (single space between "21." and "46"). NOT matches the condition.

—> After applying `normalize-space()` , becomes "21.46" . matches the condition.

⇒ Total **3 elements** will match with given Xpath

How many elements will match with XPATH= `//td[normalize-space()='21.46']`

Xpath Axes Locators -1

- **Child**

- Description: Selects all children of the current node.
- Usage: Useful for navigating to direct child elements.
- Syntax: `element = driver.find_element(By.XPATH, "//div/child::p")`
- Example: `element = driver.find_element(By.XPATH, "//div[@id='login_container']/child::input[@name='pass']")`

- **Parent**

- Description: Selects the parent of the current node.
- Usage: Useful for navigating back to the parent element.
- Syntax: `element = driver.find_element(By.XPATH, "//p/parent::div")`
- Example: `element = driver.find_element(By.XPATH, "//input[@name='pass']/parent::div")`

NOTE: For Axes example, refer <https://www.facebook.com/> website's "Password" input field

Xpath Axes Locators -2

- **Ancestor**

- Description: Selects all ancestors (parent, grandparent, etc.) of the current node.
- Usage: Useful for navigating up the hierarchy.
- Syntax: `element = driver.find_element(By.XPATH, "//span/ancestor::div")`
- Example: `element = driver.find_element(By.XPATH, "//input[@name='pass']/ancestor::div")`

- **Descendant**

- Description: Selects all descendants (children, grandchildren, etc.) of the current node.
- Usage: Useful for selecting all nested elements.
- Syntax: `element = driver.find_element(By.XPATH, "//div/descendant::span")`
- Example: `element = driver.find_element(By.XPATH, "//div[@id='login_container']/descendant::input[@name='pass']")`

Xpath Axes Locators -3

- **Following**

- Description: Selects all nodes in the document that are after current node in document order.
- Usage: Useful for selecting elements that come after a specific element.
- Syntax: `element = driver.find_element(By.XPATH, "//h1/following::p")`
- Example: `element = driver.find_element(By.XPATH, "
\"//input[@name='pass']/following::button[@name='login']")`

- **Preceding**

- Description: Selects all nodes in the document that are before current node in document order.
- Usage: Useful for selecting elements that come before a specific element.
- Syntax: `element = driver.find_element(By.XPATH, "//p/preceding::h1")`
- Example: `element = driver.find_element(By.XPATH, "
\"//button[@name='login']/preceding::input[@name='pass']")`

Xpath Axes Locators -4

- **Following-sibling**

- Description: Selects all siblings after the current node.
- Usage: Useful for selecting elements that share the same parent and come after the current element.
- Syntax: `element = driver.find_element(By.XPATH, "//h1/following-sibling::p")`
- Example: `element = driver.find_element(By.XPATH, "
 \"//input[@name='email']/following-sibling::input[@name='pass']")`

- **Preceding-sibling**

- Description: Selects all siblings before the current node.
- Usage: Useful for selecting elements that share same parent and come before current element.
- Syntax: `element = driver.find_element(By.XPATH, "//p/preceding-sibling::h1")`
- Example: `element = driver.find_element(By.XPATH, "
 \"//input[@name='pass']/preceding-sibling::input[@name='email']")`

Xpath Axes Locators -5

- **Self**

- Description: Selects the current node itself.
- Usage: Useful when you want to refer to the element you are currently working with.
- Syntax: `element = driver.find_element(By.XPATH, "///div/self::div")`
- Example: `element = driver.find_element(By.XPATH, "///input[@name='pass']/self::input")`

- **Descendant-or-self**

- Description: Selects the current node and all its descendants.
- Usage: Useful for selecting the node and everything nested within it.
- Syntax: `element = driver.find_element(By.XPATH, "///div/descendant-or-self::span")`
- Example: `element = driver.find_element(By.XPATH, "///div[@id='login_container']/descendant-or-self::input[@name='pass']")`

New Locators - Selenium4

- **above():** Finds the element above a specified element
 - `helper_ele = driver.find_element(By.ID, "password")`
`my_ele = driver.find_element(locate_with(By.TAG_NAME, "input").above(helper_ele))`
- **below():** Finds the element below a specified element
 - `helper_ele = driver.find_element(By.ID, "password")`
`my_ele = driver.find_element(locate_with(By.TAG_NAME, "input").below(helper_ele))`
- **toLeftOf():** Finds the element to the left of a specified element
 - `helper_ele = driver.find_element(By.ID, "password")`
`my_ele = driver.find_element(locate_with(By.TAG_NAME, "input").to_left_of(helper_ele))`
- **toRightOf():** Finds the element to the right of a specified element
 - `helper_ele = driver.find_element(By.ID, "password")`
`my_ele = driver.find_element(locate_with(By.TAG_NAME, "input").to_right_of(helper_ele))`
- **near():** Finds the element within 50 pixels of a specified element
 - `helper_ele = driver.find_element(By.ID, "password")`
`my_ele = driver.find_element(locate_with(By.TAG_NAME, "input").near(helper_ele))`

facebook.com Xpath Exercise

Open www.facebook.com in browser & answer below questions for “forget password?” field:

Q1] Write Xpath using text() //a[text()='Forgotten password?']

Q2] Write Xpath using normalize-space() //a[normalize-space()='Forgotten password?']

Q3] Write LINK_TEXT locator "Forgotten password?"

Q4] Write PARTIAL_LINK_TEXT locator "Forgotten"

Q5] Xpath using starts-with? //a[starts-with(@href, 'https://www.facebook.com/recover/initiate')]

Q6] Write Xpath using contains() function //a[contains(@href, 'facebook.com/recover/initiate')]

facebook.com Xpath Exercise

Open www.facebook.com in browser & answer below questions for “forget password?” field:

Q7] Write Xpath using href attribute

```
//a[@href='https://www.facebook.com/recover/initiate/?privacy_mutation_token=eyJ0eXBlljowLCJjcmVhdGlvbl90aW1lIjoxNzI1NzM2MzA2LCJjYWxsc2l0ZV9pZCI6Mzgxmjl5MDc5NTc1OTQ2fQ%3D%3D&ars=facebook_login&next']
```

Q8] Write Xpath using multiple attribute ie href & text ?

```
//a[@href='https://www.facebook.com/recover/initiate/?privacy_mutation_token=eyJ0eXBlljowLCJjcmVhdGlvbl90aW1lIjoxNzI1NzM2MzA2LCJjYWxsc2l0ZV9pZCI6Mzgxmjl5MDc5NTc1OTQ2fQ%3D%3D&ars=facebook_login&next' and text()='Forgotten password?']
```

Q9] Write absolute Xpath

```
/html[1]/body[1]/div[1]/div[1]/div[1]/div[1]/div[1]/div[1]/div[2]/div[1]/div[1]/form[1]/div[3]/a[1]
```

Q10] Xpath using DOM position? [Assuming <a> is first element with href attribute]

```
(//a[@href])[1]
```

facebook.com Xpath Exercise

Open www.facebook.com in browser & answer below questions for “forget password?” field:

Q11] Write CSS_Selector locator?

"a[href='https://www.facebook.com/recover/initiate/?privacy_mutation_token=eyJ0eXBlljowL
CJjcmVhdGlvbI90aW1lIjoxNzI1NzM2MzA2LCJjYWxsc2l0ZV9pZCI6Mzgxmjl5MDc5NTc1OTQ2f
Q%3D%3D&ars=facebook_login&next']"

Q12] Write CSS_Selector locator using partial match?

a[href*='facebook.com/recover/initiate']

Q13] Write CSS_Selector using starts-with match?

a[href^='https://www.facebook.com/recover/initiate']

Q14] Write CSS_Selector using ends-with match?

a[href\$='&ars=facebook_login&next']

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

driver = webdriver.Chrome()

driver.get("https://www.google.com/")

search_txtbox = driver.find_element(By.XPATH,
"//textarea[@name='q']")
search_txtbox.send_keys('imcc')

search_txtbox.send_keys(Keys.ENTER)
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

driver = webdriver.Chrome()

driver.get("https://www.google.com/")

multi_search_txtbox = driver.find_elements(By.XPATH,
"//textarea")

multi_search_txtbox.send_keys('imcc')
multi_search_txtbox.send_keys(Keys.ENTER)
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

driver = webdriver.Chrome()
driver.get("https://www.google.com/")

search_txtbox = driver.find_element(By.XPATH,
"//textarea[@name='q']")
search_txtbox.sendKeys('imcc')

imcc_dropdown = driver.find_element(By.XPATH,
" (//span[contains(normalize-space(), 'imcc')])[1]")

imcc_dropdown.click()
```

Selenese

- **find_element()**
 - Returns single element
 - Return type is WebElement
 - If more than one element for locator then first one picked
 - If no WebElement found then NoSuchElementException exception
 - `element = driver.find_element(By.NAME, "keshav")`
- **find_elements()**
 - Returns list of element for the locator
 - Return type is list
 - If no WebElement found then returns empty list
 - `element = driver.find_elements(By.NAME, "keshav")`

```
from selenium import webdriver  
from selenium.webdriver.common.by import By  
from selenium.webdriver.common.keys import Keys
```

```
driver = webdriver.Chrome()  
driver.get("https://www.google.com/")
```

```
single_webele = driver.find_element(By.XPATH, "//textarea")
```

```
multi_webele = driver.find_elements(By.XPATH, "//textarea")
```

```
print(type(single_webele))  
print(type(multi_webele))
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

driver = webdriver.Chrome()
driver.get("https://www.google.com/")

search_txtbox = driver.find_element(By.XPATH,
'//textarea[@name='q']')

search_txtbox.send_keys('imcc')

imcc_dropdown = driver.find_elements(By.XPATH,
'//span[contains(normalize-space(), 'imcc')]')

driver.send_keys(Keys.ENTER)
```

Selenese - Navigation

- **driver.get("www.google.com")**
 - Opens web application URL
 - Synchronous call ie command waits till url loads
- **refresh()**
 - Refreshes current page
- **forward()**
 - Goes back to next page from browser history
- **back()**
 - Goes back to previous page from browser history

Deprecated commands from previous version of Selenium:

- ~~driver.navigate().to("www.google.com")~~
- ~~driver.navigate().back()~~
- ~~driver.navigate().forward()~~

Selenese - Navigation -2

- **driver.current_url**
 - Returns current URL of web application
- **driver.title**
 - Returns title of current page
- **driver.page_source**
 - Return source code of current page

Selenese -2

- **close()**
 - closes only Current window on which Selenium is running automated tests.
 - The WebDriver session remains active, and other windows (if any) stays open
- **quit()**
 - Closes all browser windows
 - Ends the WebDriver session, releases all resources associated to webdriver

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
```

```
driver = webdriver.Chrome()
driver.get("https://www.google.com")
```

```
driver.switch_to.new_window("window")
driver.get("https://www.facebook.com")
```

```
driver.switch_to.new_window("tab")
driver.get("https://www.youtube.com")
time.sleep(10)
```

```
driver.close()    # Also try driver.quit()
```

Waits

Implicit Wait	Explicit Wait	Fluent Wait
Global wait, applicable to all webelements	Applicable to specific element	Same as Explicit wait along with polling frequency
Specific to visibility	Multiple options available to apply wait	
Waits till Element finds or Timeout happens. If element visible then don't wait for further.	Will wait till condition return false value	
Selenium's default wait	<pre>from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions</pre>	
Usage: At framework level to avoid NoSuchElementException	Usage: To handle dynamic AJAX elements	Usage: To check if any element performed specific action in an given interval
It was a feature of Watir		

Explicit Wait expected_conditions Methods

presence_of_element_located	wait until element is present in the DOM (not necessarily visible)
presence_of_all_elements_located	wait until all elements are present in the DOM
staleness_of	wait until element is no longer attached to the DOM
alert_is_present	wait until an alert is present
frame_to_be_available_and_switch_to_it	wait until frame is available and switch to it
element_to_be_selected	wait until element is selected
element_to_be_clickable	wait until element is visible and enabled
text_to_be_present_in_element	wait until text is present in element
text_to_be_present_in_element_value	wait until text is present in element's value attribute

Explicit Wait expected_conditions Methods (2)

visibility_of	wait until element is visible
visibility_of_element_located	wait until located element is visible
invisibility_of_element	wait until element is invisible
invisibility_of_element_located	wait until located element is invisible from DOM
title_is	wait until page title matches exactly
title_contains	wait until page title contains substring
url_contains	wait until URL contains substring
url_matches	wait until URL matches regex
number_of_windows_to_be(<n>)	wait until total number of windows open becomes <n>

Waits - Example

Implicit Wait	Explicit Wait	Fluent Wait
<pre>from selenium import webdriver # Initialize the WebDriver driver = webdriver.Chrome() # Set implicit wait time driver.implicitly_wait(10) # Wait for up to 10 seconds # Open a webpage driver.get("https://www.example.com") # Attempt to find an element element = driver.find_element(By.ID, "some_id") # Wait for 10 sec for the element to be found # Close the browser driver.quit()</pre>	<pre>from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC driver = webdriver.Chrome() driver.get("https://www.example.com") # Wait for up to 10 seconds wait = WebDriverWait(driver, 10) # Wait for an element to be visible element = wait.until(EC.element_to_be_clickable((By.ID, "some_id"))) # Will wait until the element is clickable element.click() driver.quit()</pre>	<pre>from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC from selenium.common.exceptions import TimeoutException driver = webdriver.Chrome() driver.get("https://www.example.com") # Create fluent wait which wait for 20 sec with polling every 2 sec wait = WebDriverWait(driver, 20, poll_frequency=2) try: # Wait for an element to be visible element = wait.until(EC.visibility_of_element_located((By.ID, "some_id"))) # Perform action on the element element.click() except TimeoutException: print("Element not found within the time limit.") driver.quit()</pre>

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://www.google.com")

driver.implicitly_wait(10)

search_box = driver.find_element(By.NAME, "q")
search_box.send_keys("Pune")
```

WAITS

CODE CONTINUED ON NEXT PAGE

continue from previous page code

wait = WebDriverWait(driver, 15)

**search_box_explicit = wait.until(
 EC.presence_of_element_located((By.NAME, "q"))))**

search_box_explicit.send_keys("Maharashtra")

**polling_wait = WebDriverWait(driver, 20, poll_frequency=3,
ignored_exceptions=[Exception])**

**search_box_polling = polling_wait.until(
 EC.visibility_of_element_located((By.NAME, "q"))))**

search_box_polling.send_keys("India")

driver.quit()

```
from selenium import webdriver    # TRY WITH INVALID XPATH #  
from selenium.webdriver.common.by import By  
from selenium.webdriver.support.ui import WebDriverWait  
from selenium.webdriver.support import expected_conditions as EC  
  
driver = webdriver.Chrome()  
driver.get("https://www.google.com")  
  
driver.implicitly_wait(10)  
  
search_box = driver.find_element(By.NAME, "wrong")    # output?  
search_box.send_keys("Pune")
```

CODE CONTINUED ON NEXT PAGE

continue from previous page code

```
wait = WebDriverWait(driver, 15)  
search_box_explicit = wait.until(  
    EC.presence_of_element_located((By.NAME, "wrong"))) # output?  
search_box_explicit.send_keys("Maharashtra")
```

```
polling_wait = WebDriverWait(driver, 20, poll_frequency=3,  
ignored_exceptions=[Exception])  
search_box_polling = polling_wait.until(  
    EC.visibility_of_element_located((By.NAME, "wrong"))) # output?  
search_box_polling.send_keys("India")
```

```
driver.quit()
```

get_attribute()

- Fetches value of attribute for given web element
- It Returns value of attribute as:
 - By default as string
 - If attribute is boolean, then it returns as true or null
 - If no attribute then it returns null

```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

get_attribute

```
driver = webdriver.Chrome()
driver.get("https://www.google.com")
search_box = driver.find_element(By.NAME, "q")

print("maxlength: ", search_box.get_attribute("maxlength"))
print("id: ", search_box.get_attribute("id"))

print("value (before typing): ", search_box.get_attribute("value"))
search_box.send_keys("IMCC")
print("value (after typing): ", search_box.get_attribute("value"))

driver.quit()
```

Checkbox + Xpath Exercise

- Checkbox tag usually starts with `<input>`
- Checkbox list values usually starts with `<type>` attribute set to “checkbox”

- HTML Example:

`<form>`

```
<input type="checkbox" id="agreeTerms"
name="terms" value="agree">
```

```
<label for="agreeTerms">I agree to the
terms and conditions</label>
```

`</form>`

Write any Xpath using ID locator?

```
//input[@id='agreeTerms']
```

Write any Xpath using value locator?

```
//input[@value='agree']
```

Write any Xpath using Name locator?

```
//input[@name='terms']
```

Write any Xpath using 1st input position?

```
//form/input[1]
```

Write any Xpath to get “I agree to the terms and conditions” element?

```
//label[text()='I agree to the terms and
conditions']/preceding-sibling::input
```

[checkbox.py](#)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
```

```
driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/checkboxes")
```

```
checkbox1 = driver.find_element(By.XPATH, "//form[@id='checkboxes']/input[1]")
checkbox2 = driver.find_element(By.XPATH, "//form[@id='checkboxes']/input[2]")
```

```
checkbox1.click()
time.sleep(5)
checkbox1.click()
time.sleep(5)
checkbox2.click()
time.sleep(5)
checkbox2.click()
time.sleep(5)
```

continue from previous page code

time.sleep(5)

if not checkbox1.is_selected(): *# Click checkbox1 to Select only if its is NOT selected*
 checkbox1.click()

time.sleep(5)

if checkbox1.is_selected(): *# Click checkbox1 to Unselect only if its selected*
 checkbox1.click()

time.sleep(5)

Dropdown + XPATH Exercise

- Dropdown tag always starts with `<select>`
- Dropdown list values usually starts with `<option>` tag

- Example:

```
<select id="myDropdown" name="university">  
  <option value="1">Pune Uni</option>  
  <option value="2">Mumbai Uni</option>  
  <option value="3">Oxford Uni</option>  
</select>
```

Write Xpath for complete dropdown?

```
//select[@id='myDropdown']
```

OR

```
//select[@name=university]
```

Write Xpath for 1st option from given dropdown?

```
//select[@id='myDropdown']/option[1]
```

Selenese -Dropdown

- Approach-1: Using SELECT class

```
from selenium.webdriver.support.select import Select
```

```
sel = Select(driver.find_element_by_xpath("//select[@name=continent]"))
```

- sel.select_by_visible_text("Europe")
- sel.select_by_index(2)
- sel.select_by_value("Europe")

*Below methods only available for multiselect dropdown

- sel.deselect_by_visible_text("Europe")
- sel.deselect_by_index(2)
- sel.deselect_by_value("Europe")
- sel.deselect_all()

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
```

```
driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/dropdown")
mydropdown = driver.find_element(By.ID, "dropdown")
dropdown = Select(mydropdown)
```

```
dropdown.select_by_visible_text("Option 1")
time.sleep(5)
```

```
selected_option = dropdown.first_selected_option
print(f"Selected option: {selected_option.text}")
dropdown.select_by_value("2")      # Select Option 2 by value
time.sleep(5)
```

```
dropdown.select_by_index(1)        # index starts from 0, here 0th position is default option
time.sleep(5)
```

<continue from previous code>

```
dropdown.deselect_by_visible_text("Option 1")    # error ?  
dropdown.deselect_all()                          # error ?
```

MULTI-SELECT DROPDOWN

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
```

```
driver = webdriver.Chrome()
driver.get("https://demoqa.com/select-menu")
multi_select_element = driver.find_element(By.ID, "cars") # The ID for Standard multi select
multi_select = Select(multi_select_element)

multi_select.select_by_visible_text("Volvo") # Add time.sleep(5)
multi_select.deselect_by_visible_text("Volvo")

multi_select.select_by_index(1) # Select "Saab" (index 1)
multi_select.select_by_value("audi") # Select "Audi"

all_selected_opt_list = multi_select.all_selected_options
for opt in all_selected_opt_list:
    print(opt.text)

multi_select.deselect_by_index(0)
multi_select.deselect_by_value("audi")
multi_select.deselect_all()
```

Selenese -Dropdown -2

- Approach-2: By using **send_keys()** method
 - However limited to selection of dropdown visible value
 - Do Not provide other action options like Select class
 - Example: myelement.send_keys("September")
- Approach-3: By storing **all XPATH options items in List** and **iterating** through it.
 - Less recommended approach
- Approach-4: By using **JavaScriptExecutor** class.
- Approach-5: By using **Actions** Class.

Write Xpath using ID attribute for “Male” radio button?

Radio Button + Xpath Exercise

//input[@id='genderMale']

- RadioButton tag starts with `<input>`
- RadioButton list values starts with `<type>` attribute set to “radio”

Write Xpath of “other” using value, name attr

//input[@name='gender' and @value='other']

Example:

```
<form>
```

```
<label for="genderMale">Male</label> <input  
type="radio" id="genderMale" name="gender"  
value="male">
```

```
<label for="genderFemale">Female</label> <input  
type="radio" id="genderFemale" name="gender"  
value="female">
```

```
<label for="genderOther">Other</label> <input  
type="radio" id="genderOther" name="gender"  
value="other">
```

```
</form>
```

For “female” radio button xpath using value attribute?

//input[@value='female']

Write Xpath for “male” using label text?

//label[text()='Male']/following-sibling::input[@type='radio']

Write Xpath for “female” using name attr?

(//input[@name='gender'])[2]

TextBox + Xpath Exercise

- Textbox tag usually starts with `<input>`
- `<type>` attribute set to `"text"`

Example Element:

```
<input type="text" class="inputtext _58mg _5dba _2ph-" data-type="text"
name="firstname" value="" aria-required="true" placeholder="" aria-label="First name"
id="u_3_8_TQ" tabindex="0" aria-describedby="js_99">
```

TEXTBOX + RADIO BUTTON

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
```

```
driver = webdriver.Chrome()
driver.get("https://www.facebook.com/")
wait = WebDriverWait(driver, 10)
```

```
create_account_button = wait.until(EC.element_to_be_clickable((By.LINK_TEXT,
"Create new account"))))
create_account_button.click()
```

```
registration_form = wait.until(EC.visibility_of_element_located((By.XPATH,
"//form[@id='reg']"))))
```

CODE CONTINUED ON NEXT PAGE

continue from previous page code

```
first_name = wait.until(EC.presence_of_element_located((By.NAME, "firstname")))
last_name = wait.until(EC.presence_of_element_located((By.NAME, "lastname")))
mobile_number = wait.until(EC.presence_of_element_located((By.NAME, "reg_email__")))
password = wait.until(EC.presence_of_element_located((By.NAME, "reg_passwd__")))
```

```
first_name.send_keys("YourName")
last_name.send_keys("YourSurname")
mobile_number.send_keys("1234567890")
password.send_keys("SecurePassword123")
```

```
day = wait.until(EC.presence_of_element_located((By.ID, "day")))
month = wait.until(EC.presence_of_element_located((By.ID, "month")))
year = wait.until(EC.presence_of_element_located((By.ID, "year")))
```

```
day.send_keys("6")
month.send_keys("Sep")
year.send_keys("1990")
```

CODE CONTINUED ON NEXT PAGE

continue from previous page code

```
gender_male = wait.until(EC.presence_of_element_located((By.XPATH,
"//input[@value='2']")))
```

```
gender_female = wait.until(EC.presence_of_element_located((By.XPATH,
"//input[@value='1']")))
```

```
gender_custom = wait.until(EC.presence_of_element_located((By.XPATH,
"//input[@value='-1']")))
```

```
gender_male.click()
```

```
sign_up_button = wait.until(EC.element_to_be_clickable((By.NAME, "websubmit")))
sign_up_button.click()
```

Either success/error message handling. Facebook wont accept automated accounts anymore

```
confirmation_message_wait = WebDriverWait(driver, 120)
```

```
confirmation_message = confirmation_message_wait.until(
    EC.presence_of_element_located((By.XPATH, "//div[contains(text(), 'confirmation')]"))
print("User created successfully.")
```

Selenese - Mouse Simulation (Action)

```
from selenium.webdriver.common.action_chains import ActionChains  
actions = ActionChains(driver)
```

- `actions.click(ele1)` -->mouse left click
 - `actions.double_click(ele1)` -->mouse left double click
 - `actions.context_click(ele1)` -->For mouse right click
- `actions.perform()` -->IMP, else no action performed

Selenese - Mouse Simulation (Action)

```
from selenium.webdriver.common.action_chains import ActionChains
```

```
actions = ActionChains(driver)
```

- actions.**move_to_element**(my_webelement) -->mouse hover
- actions.**scroll_to_element**(frame) -->scroll till frame element

```
actions.perform() -->IMP, else no action performed
```

Drag-Drop (using Action Class)

from selenium.webdriver.common.action_chains import **ActionChains**

- actions.**drag_and_drop**(my_webelement1, my_webelement2)
 - Dragging my_webelement1 to my_webelement2 on same page
- actions.**drag_and_drop_by_offset**(my_webelement, 300, 400)
 - 300=x-cordinate, 400=y-cordinate @this position my_webelement will be drag-dropped

actions.perform()

-->MOST IMP, else no action performed

```
from selenium import webdriver # MOUSE SIMULATION #
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
import time

driver = webdriver.Chrome()
driver.get("https://imcc.mespune.in/")

program_menu = driver.find_element(By.XPATH,
'//li[@id='menu-item-4383']')

actions = ActionChains(driver)
actions.move_to_element(program_menu).perform()

time.sleep(5)
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.common.action_chains import ActionChains
```

```
from selenium.webdriver.support.ui import WebDriverWait
```

```
from selenium.webdriver.support import expected_conditions as EC
```

```
import time
```

```
driver = webdriver.Chrome()
```

```
driver.set_page_load_timeout(20)
```

```
driver.maximize_window()
```

```
driver.get("https://imcc.mespune.in/")
```

```
wait = WebDriverWait(driver, 15)
```

```
actions = ActionChains(driver)
```

```
header = wait.until(
```

```
    EC.presence_of_element_located((By.XPATH, "//h2[contains(normalize-space(),  
'Prominent Recruiters')]")) )
```

CONTINUED FROM PREVIOUS PAGE CODE

```
logos = wait.until(
    EC.presence_of_all_elements_located((By.XPATH,
        "//h2[contains(normalize-space(), 'Prominent Recruiters')]/following::img")) )
print("Total logos found:", len(logos))
```

```
if len(logos) >= 3:
    third_logo = wait.until(EC.visibility_of(logos[2])) # index 2 => 3rd logo
    time.sleep(2)
    actions.click_and_hold(third_logo).move_by_offset(-100, 0).release().perform()
    # alternatively: actions.drag_and_drop_by_offset(third_logo, -100, 0).perform()
```

```
    print("Dragged 3rd logo 100px to the left.")
    time.sleep(2)
```

```
else:
    print("Less than 3 logos found")
driver.quit()
```

Drag-Drop (using Javascript)

- Javascript for drag-n-drop:

- ```
var target = arguments[0];
var dataTransfer = new DataTransfer();
var file = new File([new Uint8Array(atob(arguments[1]).split('').map(char => char.charCodeAt(0)))], 'file.txt', { type: 'text/plain' });
dataTransfer.items.add(file);
target.dispatchEvent(new DragEvent('drop', { dataTransfer }));
```

- DragEvent

- dragstart : Fired when the user starts dragging an element.
- drag : Fired repeatedly while the element is being dragged.
- dragenter : Fired when the dragged element enters a valid drop target.
- dragover : Fired when the dragged element is over a valid drop target.
- dragleave : Fired when the dragged element leaves a valid drop target.
- drop : Fired when the dragged element is dropped on a valid drop target.
- dragend : Fired when the drag operation is complete.

# Windows

- Parent and child windows
- Browser tabs
- **Windows Selenium Methods**
  - `driver.window_handles` ==>Returns List[str], IDs of all open windows
  - `driver.current_window_handle` ==>Return string of unique ID of current window
  - `driver.switch_to.window(driver.window_handles[1])`

```
from selenium import webdriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
driver = webdriver.Chrome()
driver.get("https://www.facebook.com")
wait = WebDriverWait(driver, 10)
```

```
original_window = driver.current_window_handle # Try print(original_window, type(original_window))
```

```
driver.switch_to.new_window('window')
wait.until(EC.number_of_windows_to_be(2))
```

```
driver.execute_script("window.open('http://www.google.com/');")
wait.until(EC.number_of_windows_to_be(3))
```

```
all_windows = driver.window_handles # Try print(all_windows, type(all_windows))
```

```
for window in all_windows:
```

```
 if window != original_window:
```

```
 driver.switch_to.window(window)
```

```
 break
```

```
driver.quit()
```

*# Homework: Try with 2 different drivers*

# Alerts

- Parent and child windows
- Browser tabs
- **Windows Selenium Methods**
  - `driver.window_handles` ==>Returns List[str], IDs of all open windows
  - `driver.current_window_handle` ==>Return string of unique ID of current window
  - `driver.switch_to.window(driver.window_handles[1])`

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/javascript_alerts")

confirm_button = driver.find_element(By.XPATH,
 "'//button[text()='Click for JS Confirm']")
confirm_button.click()

alert = driver.switch_to.alert # switch to the alert

print("Alert text: ", alert.text)
alert.accept() # Try alert.dismiss

result = driver.find_element(By.ID, "result")
print("Page result after accepting alert:", result.text)

driver.quit()
```

# Frames

- **Frame Types**

- Inline Frame

- Tag always starts with `<iframe>`
    - Part of same DOM

- Nested Frame

- iFrame within iFrame, creating hierarchical structure

- **How to identify frames on webpage?**

- **Frame Methods**

- `driver.switch_to.frame("id"/"name"/index/iframe_webelement)`
  - `driver.switch_to.default_content()` --> Back to main website page, coming outside frame
  - `driver.switch_to.parent_frame()` --> Back to parent frame, it may be another frame.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/iframe")

driver.switch_to.frame("mce_0_ifr") # Switch to the iframe using frame ID
print("Switched to iframe") # Homework: Try switch_to using iframe XPATH

iframe_body = driver.find_element(By.TAG_NAME, "body") # Enter text into iframe
iframe_body.send_keys("India")

driver.switch_to.default_content() # Switch back to the main document

Verify you are outside iframe and on main document
main_header = driver.find_element(By.TAG_NAME, "h3")
print("Main Document Header:", main_header.text)

driver.quit()
```

## # Nested Frames #

```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/nested_frames")
```

```
driver.switch_to.frame("frame-top") # Switch to top frame (main frame)
print("Switch to top frame")
```

```
driver.switch_to.frame("frame-left") # Switch to left frame within top frame
print("Switch to left frame")
```

```
left_frame = driver.find_element(By.TAG_NAME, "body")
print("Left Frame Text:", left_frame.text) # Get text from the left frame
```

```
driver.switch_to.parent_frame() # Switch back to the top frame
print("Switched back to top frame")
```

**# CODE CONTINUED ON NEXT PAGE**

**# continue from previous page code #**

```
driver.switch_to.frame("frame-middle") # Switch to middle frame within top frame
print("Switched to middle frame")
```

```
middle_frame = driver.find_element(By.TAG_NAME, "body")
print("Middle Frame Text: ", middle_frame.text) # Get text from middle frame
```

```
driver.switch_to.parent_frame() # Switch back to top frame
print("Switched back to top frame")
```

```
driver.switch_to.default_content()
```

```
driver.switch_to.frame("frame-bottom") # Now switch to bottom frame
bottom_frame = driver.find_element(By.TAG_NAME, "body")
print("Bottom Frame Text: ", bottom_frame.text) # Get text from bottom frame
```

```
driver.quit()
```

# Dynamic Content Handling

- Explicit Wait
  - Dynamic content may take time to load
  - Ensures dynamic content is present in DOM / visible / clickable / etc
  - Eg. `dynamic_element = wait.until(EC.presence_of_element_located((By.XPATH, "//div[@id='dynamicContent']")))`
- Cascade explicit wait with constant portion of webelement xpath using starts-with() etc
- Action on this dynamic element will like normal element once we correctly captures web element

Write Xpath to fetch “Virat” user dynamic element from HTML?

# Dynamic Content

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>IMCC College</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to the Sample Page</h1>
```

```
<div class="user" id="user-1">Keshav</div>
```

```
<div class="user" id="user-2">Potdar</div>
```

```
<div class="user" id="user-3">Virat</div>
```

```
<div class="message" id="msg-1">Message from Keshav</div>
```

```
<div class="message" id="msg-2">Message from Virat</div>
```

```
</body>
```

```
Keshav Potdar
</html>
```

```
user_with_a = wait.until(EC.presence_of_element_located((By.XPATH,
```

```
 "//div[starts-with(@id, 'user-') and starts-with(text(), 'Virat')]")))) OR
```

```
 //div[starts-with(@id, 'user-') and text()][starts-with(., 'Virat')])
```

Write Xpath to fetch “Message from Virat” user dynamic element from HTML?

```
user_with_a = wait.until(EC.presence_of_element_located((By.XPATH,
```

```
 "//div[starts-with(@class, 'message') and starts-with(text(), 'Message
from')]))))
```



Q & A

THANK YOU !

